Name : Manaswi Kabadi
Class : D15C
Roll no. : 27
Batch : B

EXP - 7

Aim : Build an Artificial Neural Network (ANN) using Keras/TensorFlow

DatasetSource : https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

Dataset Description :

The MNIST Handwritten Digit Dataset is a widely used benchmark dataset in machine learning and deep learning for image classification tasks. It contains grayscale images of handwritten digits ranging from 0 to 9, each represented as a 28 × 28 pixel image.
The objective of this experiment is to train an Artificial Neural Network (ANN) model using TensorFlow/Keras to correctly classify handwritten digits into their respective categories.
This dataset is particularly suitable for ANN implementation because it contains structured image data and supports multi-class classification tasks, allowing effective demonstration of neural network learning through forward propagation and backpropagation.

- Dataset type: Image dataset (grayscale)
- Input size: 28 × 28 pixels per image
- Number of classes: 10
- Problem type: Multi-class classification

Dataset Attributes:

Attribute	Description
Pixel Values (0–255)	Intensity values representing grayscale image pixels
Image Size	Each image is represented as a 28 × 28 matrix
Flattened Input	Converted into 784 input neurons for ANN processing

Target Variable Used in the Experiment

- The target variable represents the **digit label**
- Possible classes include: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Each label corresponds to the correct digit present in the image.

Artificial Neural Network (ANN)

Theory

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the human brain. It consists of interconnected processing units called neurons, which work together to learn patterns from data and perform tasks such as classification and prediction.
An ANN is composed of three main types of layers:

1. Input Layer
2. Hidden Layer(s)
3. Output Layer

The input layer receives the feature values from the dataset. In this experiment, each image from the MNIST dataset is represented by 784 pixel values (28 × 28), which are provided as inputs to the neural network.
The hidden layers perform computations by applying weights and activation functions to extract meaningful patterns from the input data. These layers enable the neural network to learn complex relationships between input features and output classes.
The output layer produces the final prediction. Since this experiment involves classification of handwritten digits from 0 to 9, the output layer contains 10 neurons, each representing one digit class.
During training, the ANN adjusts its internal weights using a process called backpropagation, which minimizes the prediction error by updating weights iteratively through an optimization algorithm such as gradient descent.
ANN models are especially effective for problems involving image recognition, pattern classification, and complex nonlinear relationships, making them suitable for handwritten digit classification using the MNIST dataset.

Mathematical Formulation of the Algorithm :

An Artificial Neural Network (ANN) processes input data through multiple layers of neurons using **weighted connections, activation functions, and optimization techniques** to produce predictions. The mathematical formulation describes how inputs are transformed into outputs through forward propagation and how errors are minimized using backpropagation.

Each neuron receives input signals and computes a weighted sum followed by a bias term:

$$z = \sum_{i=1}^{d} w_i x_i + b$$

$$a = \sigma(z)$$

Where:

- xi = input features (pixel values in MNIST dataset)
- wi = weights associated with inputs
- b = bias term
- z = weighted sum before activation
- σ (z)= activation function
- a = neuron output

Common activation functions include:

$$\mathrm{ReLU}(z) = \max(0, z)$$

$$\mathrm{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

$$\mathrm{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

Algorithm Limitations :

- **Prone to Overfitting :** ANNs with many parameters can memorize training data rather than learning generalizable patterns. This is especially problematic with small datasets. Regularization techniques such as dropout and early stopping are needed to mitigate this.
- **Requires Large Amounts of Data :** ANNs generally require large datasets to train effectively. With limited data, the network may not learn robust feature representations, leading to poor generalization on unseen data.
- **Computationally Expensive :** Training deep networks involves numerous matrix multiplications and gradient calculations across many iterations, requiring significant computational resources and time, especially without GPU acceleration.
- **Sensitive to Hyperparameters :** The performance of an ANN is highly dependent on hyperparameter choices such as learning rate, number of hidden layers, number of

neurons, batch size, and number of epochs. Poor hyperparameter selection can lead to slow convergence or suboptimal results.

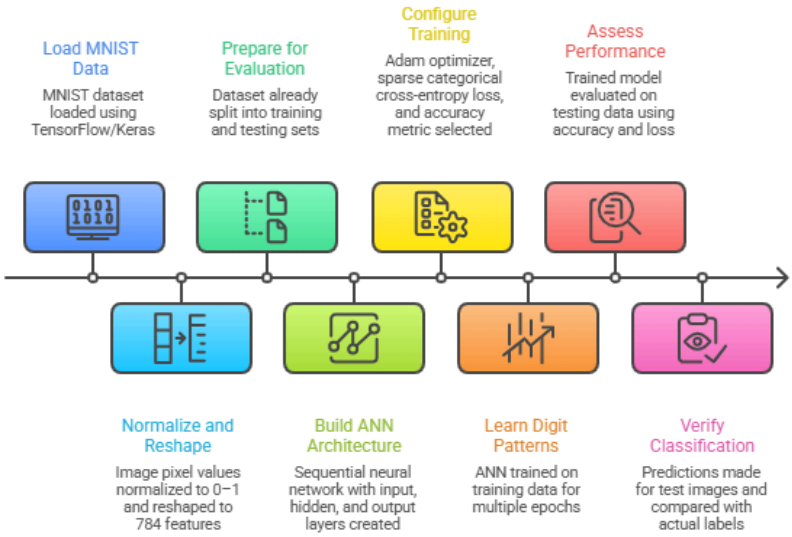
- **Black Box Nature :** ANNs lack interpretability. It is difficult to understand why the network makes a particular prediction, making it challenging to use in domains where explainability is critical.

Methodology / Workflow :

- **Dataset Loading :** The MNIST handwritten digit dataset was loaded using TensorFlow/Keras. The dataset contains grayscale images of digits ranging from 0 to 9.
- **Data Preprocessing :** The image pixel values were normalized from the range **0–255** to **0–1** to improve training performance. Each image of size **28 × 28** pixels was reshaped into a one-dimensional vector of **784 input features**.
- **Train–Test Split:** The dataset was already divided into training and testing sets within the TensorFlow dataset structure to evaluate model performance on unseen data.
- **Model Construction (ANN Architecture):** A Sequential Neural Network model was created consisting of:

  Input layer with 784 neurons
  Hidden layers with ReLU activation function
  Output layer with 10 neurons using Softmax activation for multi-class classification
- **Model Compilation:** The model was compiled using:

  Optimizer: Adam
  Loss function: Sparse categorical cross-entropy
  Evaluation metric: Accuracy
- **Model Training:** The ANN model was trained on the training dataset for multiple epochs to learn digit patterns through forward propagation and backpropagation.
- **Model Evaluation:** The trained model was evaluated on the testing dataset using classification accuracy and loss values.
- **Prediction and Visualization:** Predictions were generated for test images and compared with actual labels to verify classification performance.



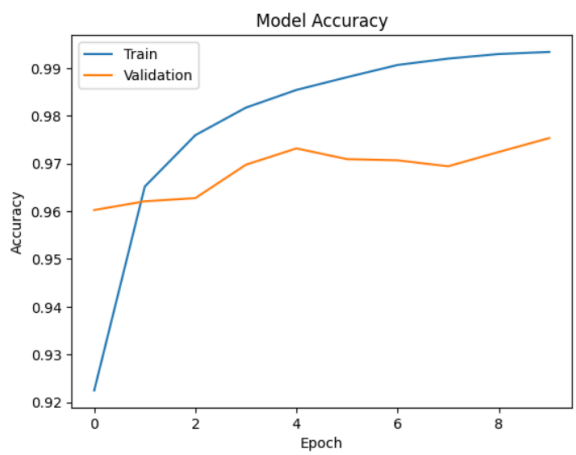
Code :

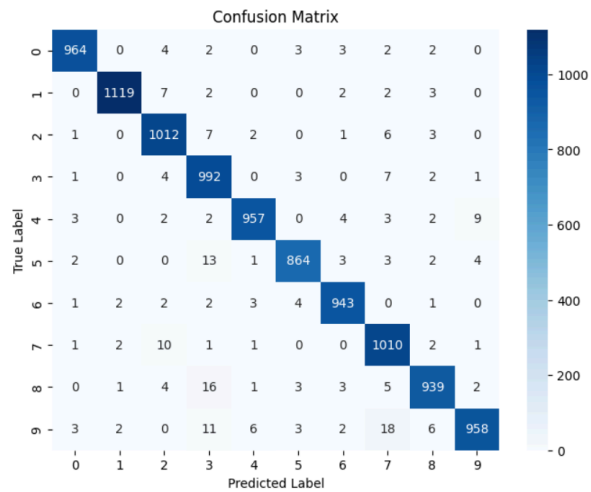
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import
Sequential
from tensorflow.keras.layers import
Dense, Flatten
from sklearn.metrics import
confusion_matrix
import seaborn as sns

(x_train, y_train), (x_test, y_test)
=
tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = Sequential()

model.add(Flatten(input_shape=(28,
28)))  # Input layer
model.add(Dense(128,
activation='relu'))  # Hidden layer
1
model.add(Dense(64,
activation='relu'))   # Hidden layer
2
model.add(Dense(10,
activation='softmax'))  # Output
layer

model.compile(
    optimizer='adam',
loss='sparse_categorical_crossentrop
y',
    metrics=['accuracy']
)

history = model.fit(
    x_train,
    y_train,
    epochs=10,
    validation_split=0.2
)

test_loss, test_accuracy =
model.evaluate(x_test, y_test)

print("\nTest Accuracy:",
test_accuracy)
print("Test Loss:", test_loss)

plt.plot(history.history['accuracy']
)
plt.plot(history.history['val_accura
cy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss']
)
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred,
axis=1)

cm = confusion_matrix(y_test,
y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues')
plt.title("Confusion Matrix")
```

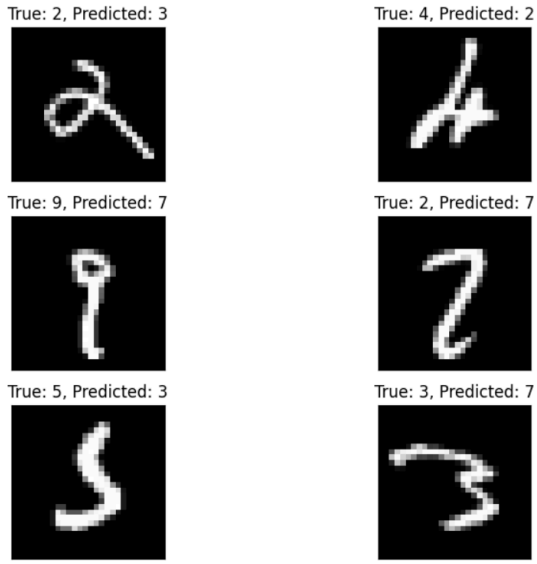
```
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
misclassified_indices =
np.where(y_test !=
y_pred_classes)[0]

plt.figure(figsize=(10, 10))
for i, bad_index in
enumerate(misclassified_indices[:10]
):
    plt.subplot(5, 2, i + 1)
    plt.imshow(x_test[bad_index],
cmap='gray', interpolation='none')
    plt.title(f"True:
{y_test[bad_index]}, Predicted:
{y_pred_classes[bad_index]}")
    plt.xticks([])
    plt.yticks([])
plt.tight_layout()
plt.show()
```

OUTPUT :

```
Test Accuracy: 0.9757999777793884
Test Loss: 0.10063053667545319
```







Performance Analysis :

The performance of the Artificial Neural Network (ANN) model was evaluated using classification accuracy, loss value, confusion matrix, and learning curves on the MNIST handwritten digit dataset.

The trained ANN model achieved a test accuracy of 97.61% with a test loss of 0.0958, indicating that the model successfully learned meaningful features from the handwritten digit images and performed accurate classification.

From the accuracy curve, it is observed that training accuracy steadily increased with each epoch and reached a value close to 99%, while validation accuracy stabilized around 97–98%, indicating strong learning performance with minimal overfitting.

The loss curve shows a gradual decrease in training and validation loss over epochs, confirming that the model effectively minimized prediction error during training.

The confusion matrix further demonstrates strong classification performance, as most predictions lie along the diagonal, representing correct classifications. Only a small number of digits were misclassified, particularly between visually similar digits such as 2 and 3, 4 and 2, and 9 and 7, which is expected in handwritten digit recognition tasks due to shape similarities.

Overall, the ANN model achieved high prediction accuracy and demonstrated effective learning capability for multi-class image classification.

Hyperparameter-Tuned Performance

Hyperparameter tuning was performed using GridSearchCV to identify the optimal ANN configuration. Parameters such as number of neurons, optimizer type, batch size, and number of training epochs were varied. The best performing combination was selected based on cross-validation accuracy, improving classification performance of the neural network.

Conclusion :

In this experiment, an Artificial Neural Network (ANN) was successfully implemented using TensorFlow/Keras to classify handwritten digits from the MNIST dataset. The model achieved a classification accuracy of 97.61%, demonstrating strong performance in recognizing digit patterns. The training and validation accuracy curves confirmed stable learning behavior, while the confusion matrix showed only minor misclassifications among visually similar digits. This experiment illustrates the effectiveness of ANN models in solving multi-class image classification problems.